

Calling all who code. [Take the 2023 Developer Survey.](#)

Node.js spawn child process and get terminal output live

Asked 10 years, 4 months ago Modified 7 months ago Viewed 200k times



168



I have a script that outputs 'hi', sleeps for a second, outputs 'hi', sleeps for 1 second, and so on and so forth. Now I thought I would be able to tackle this problem with this model.

```
var spawn = require('child_process').spawn,  
    temp   = spawn('PATH TO SCRIPT WITH THE ABOVE BEHAVIOUR');  
  
temp.stdout.pipe(process.stdout);
```

Now the problem is that the task needs to be finished in order for the output to be displayed. As I am understanding it, this is due to the fact that the newly spawned process takes execution control. Obviously node.js does not support threads so any solutions? My idea was to possibly run two instances, first one for the specific purpose of creating the task and have it pipe the output to process of the second instance, considering this can be achieved.

node.js

spawn

capture-output

Share Improve this question Follow

edited Jan 23, 2017 at 21:39



Da Dan Dascalescu

Dasca 140k ● 51 ● 312 ● 402

asked Jan 15, 2013 at 7:09



foklepoint

2,181 ● 2 ● 15 ● 15

5 If child process is written `python` then do not forget to pass `-u` flag for it to not buffer console output, otherwise it will look like script is not live stackoverflow.com/a/49947671/906265 – Ivar Nov 13, 2018 at 7:21

Use npmjs.com/package/cross-spawn instead of anything else. It's just better. – Andrew Koster Mar 17, 2020 at 23:06

@foklepoint Did you manage to get it right in the end? I am currently facing a similar problem and none of the answers seem to work for me – davidthorand May 28, 2022 at 17:59

10 Answers

Sorted by:

Highest score (default)



It's much easier now (6 years later)!

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

[Sign up](#)

- Class: ChildProcess
 - Event: 'error'
 - Event: 'exit'
 - Event: 'close'
 - Event: 'disconnect'
 - Event: 'message'

There are also a bunch of **objects from childObject**, they are:

- Class: ChildProcess
 - child.stdin
 - child.stdout
 - child.stderr
 - child.stdio
 - child.pid
 - child.connected
 - child.kill([signal])
 - child.send(message[, sendHandle][, callback])
 - child.disconnect()

See more information here about childObject: https://nodejs.org/api/child_process.html

Asynchronous

If you want to run your process in the background while node is still able to continue to execute, use the asynchronous method. You can still choose to perform actions after your process completes, and when the process has any output (for example if you want to send a script's output to the client).

[child_process.spawn\(...\)](#); (Node v0.1.90)

```
var spawn = require('child_process').spawn;
var child = spawn('node ./commands/server.js');

// You can also use a variable to save the output
// for when the script closes later
var scriptOutput = "";

child.stdout.setEncoding('utf8');
child.stdout.on('data', function(data) {
```

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up



```

        data=data.toString();
        scriptOutput+=data;
    });

    child.stderr.setEncoding('utf8');
    child.stderr.on('data', function(data) {
        //Here is where the error output goes

        console.log('stderr: ' + data);

        data=data.toString();
        scriptOutput+=data;
    });

    child.on('close', function(code) {
        //Here you can get the exit code of the script

        console.log('closing code: ' + code);

        console.log('Full output of script: ',scriptOutput);
    });

```

Here's **how you would use a callback + asynchronous method**:

```

var child_process = require('child_process');

console.log("Node Version: ", process.version);

run_script("ls", ["-l", "/home"], function(output, exit_code) {
    console.log("Process Finished.");
    console.log('closing code: ' + exit_code);
    console.log('Full output of script: ',output);
});

console.log ("Continuing to do node things while the process runs at the same time...");

// This function will output the lines from the script
// AS is runs, AND will return the full combined output
// as well as exit code when it's done (using the callback).
function run_script(command, args, callback) {
    console.log("Starting Process.");
    var child = child_process.spawn(command, args);

    var scriptOutput = "";

    child.stdout.setEncoding('utf8');
    child.stdout.on('data', function(data) {
        console.log('stdout: ' + data);

        data=data.toString();
        scriptOutput+=data;
    });

    child.stderr.setEncoding('utf8');
    child.stderr.on('data', function(data) {

```

```
});  
  
child.on('close', function(code) {  
    callback(scriptOutput,code);  
});  
}
```

Using the method above, you can send every line of output from the script to the client (for example using Socket.io to send each line when you receive events on `stdout` or `stderr`).

Synchronous

If you want node to stop what it's doing and **wait until the script completes**, you can use the synchronous version:

[child_process.spawnSync\(...\)](#); (Node v0.11.12+)

Issues with this method:

- If the script takes a while to complete, your server will hang for that amount of time!
- **The stdout will only be returned once the script has finished running.** Because it's synchronous, it cannot continue until the current line has finished. Therefore it's unable to capture the 'stdout' event until the spawn line has finished.

How to use it:

```
var child_process = require('child_process');  
  
var child = child_process.spawnSync("ls", ["-l", "/home"], { encoding : 'utf8' });  
console.log("Process finished.");  
if(child.error) {  
    console.log("ERROR: ",child.error);  
}  
console.log("stdout: ",child.stdout);  
console.log("stderr: ",child.stderr);  
console.log("exit code: ",child.status);
```

Share Improve this answer Follow

edited Aug 9, 2019 at 17:59

answered Sep 30, 2015 at 17:57



Katie

44.9k ● 18 ● 92 ● 123

18 +1, this should be chosen as the right answer now. Just a note, the data variable in the callback comes in as Buffer object. You can use `child.stdout.setEncoding('utf8')` if you want utf8 strings coming in.

– Ashish Feb 17, 2016 at 16:54

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up



- 2 Hey @ChristianHujer ! I updated my answer to include both async and sync :D – [Katie](#) Aug 9, 2019 at 18:00
- if you have a script that is: `console.log("Output 1"); console.error("Boom"); console.log("Output 2");` and I'm doing `spawnAsync('node ./script.js')` ... how do you preserve the order of the output? My output always seems to come out in the incorrect order. – [Bryan Ray](#) Apr 24, 2020 at 22:23
- 1 If you're using `spawnSync` and want the values to come back as a string instead of a buffer you'll want to feed in `{ encoding: 'utf-8' }` as part of the options (3rd param). – [K. Waite](#) Jul 18, 2020 at 19:08



116



I'm still getting my feet wet with Node.js, but I have a few ideas. first, I believe you need to use `execFile` instead of `spawn`; `execFile` is for when you have the path to a script, whereas `spawn` is for executing a well-known command that Node.js can resolve against your system path.

1. [Provide a callback](#) to process the buffered output:

```
var child = require('child_process').execFile('path/to/script', [
  'arg1', 'arg2', 'arg3',
], function(err, stdout, stderr) {
  // Node.js will invoke this callback when process terminates.
  console.log(stdout);
});
```

2. Add a listener to the child process' stdout [stream](#) ([9thport.net](#))

```
var child = require('child_process').execFile('path/to/script', [
  'arg1', 'arg2', 'arg3' ]]);
// use event hooks to provide a callback to execute when data are available:
child.stdout.on('data', function(data) {
  console.log(data.toString());
});
```

Further, there appear to be options whereby you can detach the spawned process from Node's controlling terminal, which would allow it to run asynchronously. I haven't tested this yet, but there are examples in the [API docs](#) that go something like this:

```
child = require('child_process').execFile('path/to/script', [
  'arg1', 'arg2', 'arg3',
], {
  // detachment and ignored stdin are the key here:
  detached: true,
  stdio: [ 'ignore', 1, 2 ]
});
// and unref() somehow disentangles the child's event loop from the parent's:
child.unref();
child.stdout.on('data', function(data) {
  console.log(data.toString());
});
```

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

[Sign up](#)



hayesgm

8,610 ● 1 ● 38 ● 40



RubyTuesdayDONO

2,329 ● 2 ● 25 ● 37

9 Bonus points if you can explain how to do this with `exec()` as I need to execute a shell cmd. – [DynamicDan](#) Apr 10, 2014 at 19:35

3 You can use `child.spawn()` with the `shell` option set to `true`. [nodejs.org/api/...](#) – [CedX](#) Jun 2, 2016 at 14:35

12 You can also pipe `child.stdout` directly to `process.stdout` with `child.stdout.pipe(process.stdout);` – [darkadept](#) Oct 19, 2016 at 20:43

```
@DynamicDan javascript let childProcess = exec (    './script-to-run --arg1 arg1value',
( error, stdout, stderr ) =>    {    console.log( '[CALLBACK]: ' + error ); // or
stdout or stderr    } ); // Same as with spawn: childProcess.stdout.on (    'data',
( data ) =>    {    console.log( '[LIVE]: ' + data ); // Here's your live data!    }
); – Rik Jun 8, 2018 at 14:05
```

Here is the cleanest approach I've found:

54

```
require("child_process").spawn('bash', ['./script.sh'], {
  cwd: process.cwd(),
  detached: true,
  stdio: "inherit"
});
```

Share Improve this answer Follow

edited Feb 26, 2020 at 7:18

answered Nov 16, 2017 at 20:26



AmerllicA

Amer 27.9k ● 15 ● 126 ● 152



Harel Ashwal

1,420 ● 1 ● 13 ● 11

45 What is it doing, exactly? Why does it work? Why is this the cleaner approach? – [raisinrising](#) Sep 16, 2019 at 5:42

@raisinrising It is setting the process to inherit the stdio handles (including stdout). It is cleaner because there is only one function invocation (sorry for the partial comment that I deleted, a pack of index cards was dropped on my keyboard while I was away) – [PoolloverNathan](#) Apr 16, 2022 at 2:14

Simple and worked for me. I didnt need the detached option and my process was actually attached 😊 – [Pierre](#) Jan 6 at 7:54

I had a little trouble getting logging output from the "npm install" command when I spawned npm in a child process. The realtime logging of dependencies did not show in the parent console.

19

The simplest way to do what the original poster wants seems to be this (spawn npm on windows and log everything to parent console):

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up



```
var options = {
  stdio: 'inherit' //feed all child process logging into parent process
};

var childProcess = spawn('npm.cmd', args, options);
childProcess.on('close', function(code) {
  process.stdout.write('"npm install" finished with code ' + code + '\n');
});
```

Share Improve this answer Follow

answered Feb 5, 2016 at 2:05



agenaille

agen: 211 ● 2 ● 3

PHP-like passthru

7

```
import { spawn } from 'child_process';

export default async function passthru(exe, args, options) {
  return new Promise((resolve, reject) => {
    const env = Object.create(process.env);
    const child = spawn(exe, args, {
      ...options,
      env: {
        ...env,
        ...options.env,
      },
    });
    child.stdout.setEncoding('utf8');
    child.stderr.setEncoding('utf8');
    child.stdout.on('data', data => console.log(data));
    child.stderr.on('data', data => console.log(data));
    child.on('error', error => reject(error));
    child.on('close', exitCode => {
      console.log('Exit code:', exitCode);
      resolve(exitCode);
    });
  });
}
```

Usage

```
const exitCode = await passthru('ls', ['-al'], { cwd: '/var/www/html' })
```

Share Improve this answer Follow

answered Jul 27, 2020 at 23:12



Guest

71 ● 1 ● 1

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up



```
setInterval(function() {  
    process.stdout.write("hi");  
}, 1000); // or however else you want to run a timer
```

parent:

```
require('child_process').fork('./childfile.js');  
// fork'd children use the parent's stdio
```

Share Improve this answer Follow

answered Aug 13, 2013 at 2:14



Sandro Pasquali

393 ● 2 ● 9

I found myself requiring this functionality often enough that I packaged it into a library called [std-pour](#). It should let you execute a command and view the output in real time. To install simply:

```
npm install std-pour
```

Then it's simple enough to execute a command and see the output in realtime:

```
const { pour } = require('std-pour');  
pour('ping', ['8.8.8.8', '-c', '4']).then(code => console.log(`Error Code: ${code}`));
```

It's promised based so you can chain multiple commands. It's even function signature-compatible with [child_process.spawn](#) so it should be a drop in replacement anywhere you're using it.

Share Improve this answer Follow

answered Jun 17, 2017 at 18:20



Joel B

11.8k ● 10 ● 61 ● 68

1 @KodieGrantham glad it's working for you! Y'all seem like you're doing some cool work so I hope it keeps you running. – Joel B Sep 25, 2019 at 15:40

Adding a sample for `exec` as I too had needed live feedback and wasn't getting any until after the script finished. `exec` does return an **EventEmitter**, contrary to the many claims that only `spawn` works in such a way.

This supplements the comment I made to the accepted answer more thoroughly.

The interface for `exec` is similar to `spawn`:

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up



```
// INCLUDES
import * as childProcess from 'child_process'; // ES6 Syntax

// DEFINES
let exec = childProcess.exec; // Use 'var' for more proper
                                // semantics, or 'const' it all
                                // if that's your thing; though 'let' is
                                // true-to-scope;

// Return an EventEmitter to work with, though
// you can also chain stdout too:
// (i.e. exec( ... ).stdout.on( ... ); )
let childProcess = exec
(
  './binary command -- --argument argumentValue',
  ( error, stdout, stderr ) =>
  {
    // When the process completes:
    if( error )
    {
      console.log( `${error.name}: ${error.message}` );
      console.log( `[STACK] ${error.stack}` );
    }

    console.log( stdout );
    console.log( stderr );
    callback(); // Gulp stuff
  }
);
```

Now its as simple as registering an event handler for `stdout`:

```
childProcess.stdout.on( 'data', data => console.log( data ) );
```

And for `stderr`:

```
childProcess.stderr.on( 'data', data => console.log( `[ERROR]: ${data}` ) );
```

You can also `pipe` stdout to the main process' stdout:

```
childProcess.stdout.pipe( process.stdout );
```

Not too bad at all - HTH

Share Improve this answer Follow

edited Apr 14, 2021 at 17:29

answered Jun 8, 2018 at 14:45



Rik
606 ● 6 ● 11

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up



1

```
import { spawn } from 'node:child_process'
import process from 'node:process'

const script = spawn('path/to/script', { stdio: 'inherit' })
script.on('close', process.exit)
```

[Share](#) [Improve this answer](#) [Follow](#)

answered Sep 21, 2022 at 22:39

[cmolina](#)[cmoli](#) 833 ● 1 ● 11 ● 21

-1



I ran into a situation where none of the above worked when I was spawning a Python 3 script. I would get data from stdout, but only once the child terminated.

As it turns out, Python buffers stdout by default. It's possible to disable stdout buffering by including `-u` as a command line parameter to python3.

[Share](#) [Improve this answer](#) [Follow](#)

answered Sep 22, 2021 at 20:45

[pward123](#)

734 ● 6 ● 9